Manual Tecnico
Generador de Analizadores
Wison

Backend

Wison.jison
El archivo `wison.jison` define la gramática del lenguaje Wison utilizando la herramienta Jison (equivalente a Flex + CUP para JavaScript). Este archivo es el punto de entrada del sistema: recibe código Wison escrito por el usuario y produce un AST (Árbol de Sintaxis Abstracta) que contiene toda la información necesaria para construir un analizador LL(1). Jison es un generador de parsers LALR(1) para JavaScript. A partir del archivo **`.jison`**, genera un archivo **`.js`** que funciona como el parser compilado. Este parser se invoca con **`parser.parse(texto)`** y retorna el AST resultante.

Compilación
Para generar el parser a partir de la gramática se ejecuta:
npx jison wison.jison -o wison.js

Esto produce **`wison.js`** que es el parser listo para usar en Node.js. Este archivo no se edita manualmente — cada vez que se modifica `wison.jison` se debe recompilar.
Sección léxica (Lexer)
La sección léxica define los tokens que el parser reconoce en el código Wison. Cada regla es una expresión regular que cuando hace match retorna un token con un nombre específico.

Elementos ignorados
El lexer ignora tres tipos de contenido que no son significativos para el análisis:
- Espacios en blanco, tabulaciones y saltos de línea: se descartan con la regla **`\s+`**
- Comentarios de línea: todo lo que va después del carácter **`#`** hasta el final de la línea, reconocido con **`"#"[^\n]*`**
- Comentarios de bloque: todo lo que va entre **`/**`** y **`*/`**, reconocido con **`\/\*\*[\s\S]*?\*\/`.**
Se usa **`[\s\S]*?`** para que cierre en el primer **`*/`** que encuentre

Delimitadores de estructura
Estos tokens delimitan las secciones del lenguaje Wison. Se reconocen los más largos primero para evitar conflictos (por ejemplo `{{:` antes que `{:`):

- **`{{:`** → token **ABRE_SYNTAX**. Abre el bloque de definiciones sintácticas.
- **`:}}`** → token **CIERRA_SYNTAX**. Cierra el bloque sintáctico.
- **`{:`** → token **ABRE_LEX**. Abre el bloque de definiciones léxicas.
- **`:}`** → token **CIERRA_LEX**. Cierra el bloque léxico.
- **`?Wison`** → token **CIERRA_WISON**. Cierra toda la estructura Wison. Se reconoce como un solo token para evitar conflicto con el operador `?`.
- **`¿`** → token **ABRE_WISON**. Abre la estructura Wison después de la palabra reservada **Wison**.

Palabras reservadas
Son case-sensitive, es decir que `Terminal` es válido pero `terminal` no:
- **`Wison`** → token **WISON**. Indica el inicio de la estructura.
- **`Lex`** → token **LEX**. Indica el inicio del bloque léxico.

- `**Syntax**` → token **SYNTAX**. Indica el inicio del bloque sintáctico.
- `**Terminal**` → token **TERMINAL**. Palabra clave para declarar un terminal.
- `**No_Terminal**` → token **NO_TERMINAL**. Palabra clave para declarar un no terminal.
- `**Initial_Sim**` → token **INITIAL_SIM**. Palabra clave para declarar el símbolo inicial.

Operadores y símbolos
- `**<-**` → token **FLECHA_LEX**. Operador de asignación en declaraciones de terminales.
- `**<=**` → token **FLECHA_PROD**. Operador de definición de producciones.
- `**;**` → token **PUNTO_COMA**. Terminador de sentencia.
- `|` → token **PIPE**. Separador de alternativas en producciones.
- `**(**` → token **PAREN_IZQ**. Paréntesis de apertura para agrupación en expresiones léxicas.
- `**)**` → token **PAREN_DER**. Paréntesis de cierre.
- `*` → token **ASTERISCO**. Operador de Kleene (cero o más repeticiones).
- `**+**` → token **MAS_OP**. Operador de cerradura positiva (una o más repeticiones).
- `**?**` → token **INTERROGACION**. Operador opcional (cero o una repetición).

Rangos especiales
Solo se reconocen estos dos patrones de rango según la especificación del enunciado:
- `**[aA-zZ]**` → token **RANGO_AZ**. Representa cualquier letra mayúscula o minúscula.
- `**[0-9]**` → token **RANGO_09**. Representa cualquier dígito del 0 al 9.

Literales entre comillas simples
El patrón `**\'[^']+\'**` reconoce cualquier secuencia de uno o más caracteres encerrados entre comillas simples. La acción `**yytext = yytext.slice(1,-1)**` elimina las comillas y deja solo el contenido. Ejemplos: `**'a'**` produce el valor `**a**`, `**'FIN'**` produce el valor `**FIN**`, `**'+'**` produce el valor `**+**`. Retorna el token **STRING**.

Identificadores con prefijo
- `**$_**` seguido de letras, números o guión bajo → token **DOLLAR_ID**. Identifica **terminales**. Ejemplos: `**$_A**`, `**$_Mas**`, `**$_NUMERO_2**`.
- `**%_**` seguido de letras, números o guión bajo → token **PERCENT_ID**. Identifica **no terminales**. Ejemplos: `**%_S**`, `**%_Prod_A**`, `**%_EXPRESION_P**`.

Fin de archivo
- `**<<EOF>>**` → token EOF. Indica que se llegó al final del texto de entrada.

Manejo de errores léxicos
La última regla del lexer captura cualquier carácter que no haya sido reconocido por las reglas anteriores. En vez de detener el análisis, acumula los caracteres no reconocidos consecutivos en un solo mensaje de error. Si el carácter no reconocido está en la misma línea que el error anterior, se concatena al texto del error existente. Esto produce mensajes como `Texto no reconocido "@invalido" en línea 4` en vez de un error por cada carácter individual.

Los errores se almacenan en el array `**erroresLexicos**` que se incluye en el AST de salida para que las fases posteriores puedan reportarlos al usuario.

Sección sintáctica (Parser)
La sección sintáctica define las reglas gramaticales que determinan la estructura válida de un archivo Wison. Cada regla tiene acciones semánticas (código JavaScript entre llaves) que construyen el AST.

Estructura principal: programa
programa → **WISON ABRE_WISON bloque_lexico bloque_sintactico CIERRA_WISON EOF**

Es la regla raíz. Espera la secuencia `**Wison ¿ ... ?Wison**` con el bloque léxico y el bloque sintáctico adentro. La acción semántica construye el objeto AST final con cuatro campos: terminales (del bloque léxico), noTerminales, simboloInicial y producciones (del bloque sintáctico). También incluye los arrays de erroresLexicos y erroresSintacticos.

Tiene una regla de recuperación de errores: si falta el `?Wison` de cierre, captura el error con el token especial `**error**` y reporta el problema sin detener el análisis.

Bloque léxico: bloque_lexico
bloque_lexico → LEX ABRE_LEX lista_declaraciones_terminales CIERRA_LEX
| LEX ABRE_LEX CIERRA_LEX

Espera `**Lex {:**` seguido de una o más declaraciones de terminales y cierra con `**:}**`. La segunda alternativa permite un bloque léxico vacío.

Lista de declaraciones de terminales: lista_declaraciones_terminales
lista_declaraciones_terminales → lista_declaraciones_terminales declaracion_terminal
| declaracion_terminal

Declaración de terminal: declaracion_terminal
declaracion_terminal → TERMINAL DOLLAR_ID FLECHA_LEX expresion_regular PUNTO_COMA
| TERMINAL error PUNTO_COMA

La primera alternativa es la declaración correcta: `**Terminal $_NOMBRE <- expresión ;**`. Produce un objeto con el nombre del terminal y su expresión.

La segunda alternativa es recuperación de errores: si algo falla entre `Terminal` y `;`, el parser descarta tokens hasta encontrar `;`, reporta el error y continúa con la siguiente declaración.

Expresión regular: expresion_regular
expresion_regular → expresion_regular elemento_con_operador
| elemento_con_operador
```

Define la concatenación implícita. Cuando dos elementos aparecen uno junto al otro, se concatenan. Produce nodos de tipo `concatenacion` con hijos `izq` y `der`.
```

Elemento con operador: elemento_con_operador

elemento_con_operador → elemento_base ASTERISCO
| elemento_base MAS_OP
| elemento_base INTERROGACION
| elemento_base

Aplica los operadores unarios de expresiones regulares a un elemento base. Produce nodos de tipo `kleene` (para `*`), `positiva` (para `+`) u `opcional` (para `?`), cada uno con un campo `expresion` que apunta al elemento base.

Elemento base: elemento_base

elemento_base → STRING
| RANGO_AZ
| RANGO_09
| DOLLAR_ID
| PAREN_IZQ expresion_regular PAREN_DER

Son los elementos atómicos de una expresión léxica:
- STRING produce un nodo `{ tipo: 'string', valor: '...' }` con el texto entre comillas.
- RANGO_AZ produce un nodo `{ tipo: 'rango', valor: 'aA-zZ' }`.
- RANGO_09 produce un nodo `{ tipo: 'rango', valor: '0-9' }`.
- DOLLAR_ID produce un nodo `{ tipo: 'referencia', valor: '$_...' }` que apunta a otro terminal.
- Los paréntesis agrupan una subexpresión y producen un nodo `{ tipo: 'grupo', expresion: ... }`.

Bloque sintáctico: bloque_sintactico

bloque_sintactico → SYNTAX ABRE_SYNTAX contenido_bloque_sintactico CIERRA_SYNTAX
| SYNTAX ABRE_SYNTAX CIERRA_SYNTAX

Espera `Syntax {{:` seguido del contenido y cierra con `:}}`. Permite bloque vacío.

Contenido del bloque sintáctico: contenido_bloque_sintactico

contenido_bloque_sintactico → contenido_bloque_sintactico elemento_sintactico
| elemento_sintactico

Acumula los elementos en un objeto con tres campos: noTerminales (array), simboloInicial (string) y producciones (objeto).

Elemento sintáctico: elemento_sintactico

elemento_sintactico → declaracion_no_terminal
| declaracion_simbolo_inicial

| produccion

Cada elemento se etiqueta con su tipo (`no_terminal`, `simbolo_inicial` o `produccion`) para que la regla padre sepa cómo acumularlo.

Declaración de no terminal: declaracion_no_terminal

declaracion_no_terminal → NO_TERMINAL PERCENT_ID PUNTO_COMA
| NO_TERMINAL error PUNTO_COMA

Reconoce `No_Terminal %_Nombre ;`. La segunda alternativa es recuperación de errores.

Declaración de símbolo inicial: declaracion_simbolo_inicial

declaracion_simbolo_inicial → INITIAL_SIM PERCENT_ID PUNTO_COMA
| INITIAL_SIM error PUNTO_COMA

Reconoce `**Initial_Sim %_Nombre ;**`. La segunda alternativa es recuperación de errores.

Producción: produccion

produccion → PERCENT_ID FLECHA_PROD cuerpo_produccion PUNTO_COMA
| PERCENT_ID error PUNTO_COMA

Reconoce `%_Nombre <= cuerpo ;`. Produce un objeto con el nombre del no terminal y un array de alternativas.

Cuerpo de producción: cuerpo_produccion

cuerpo_produccion → cuerpo_produccion PIPE alternativa
| alternativa

Separa las alternativas por `|` y las acumula en un array.

Alternativa: alternativa

alternativa → lista_simbolos
| (vacío)

Una alternativa puede ser una secuencia de símbolos o vacía (épsilon). La alternativa vacía produce el array `['ε']`.

Lista de símbolos: lista_simbolos

lista_simbolos → lista_simbolos simbolo
| simbolo

Acumula los símbolos de una alternativa en un array.

Símbolo: simbolo

simbolo → PERCENT_ID
| DOLLAR_ID

Un símbolo en una producción puede ser un no terminal (`%_Nombre`) o un terminal (`$_Nombre`).

Estructura del AST generado
El parser retorna un objeto JavaScript con la siguiente estructura:

```
{
  terminales: {
    "$_Nombre": { tipo:
"string|rango|kleene|positiva|opcional|concatenacion|grupo|referencia", ... },
    ...
  },
  noTerminales: ["%_S", "%_E", ...],
  simboloInicial: "%_S",
  producciones: {
    "%_S": [["$_A", "%_E"], ["ε"]],
    ...
  },
  erroresLexicos: [{ tipo, mensaje }],
  erroresSintacticos: [{ tipo, mensaje }]
}
```

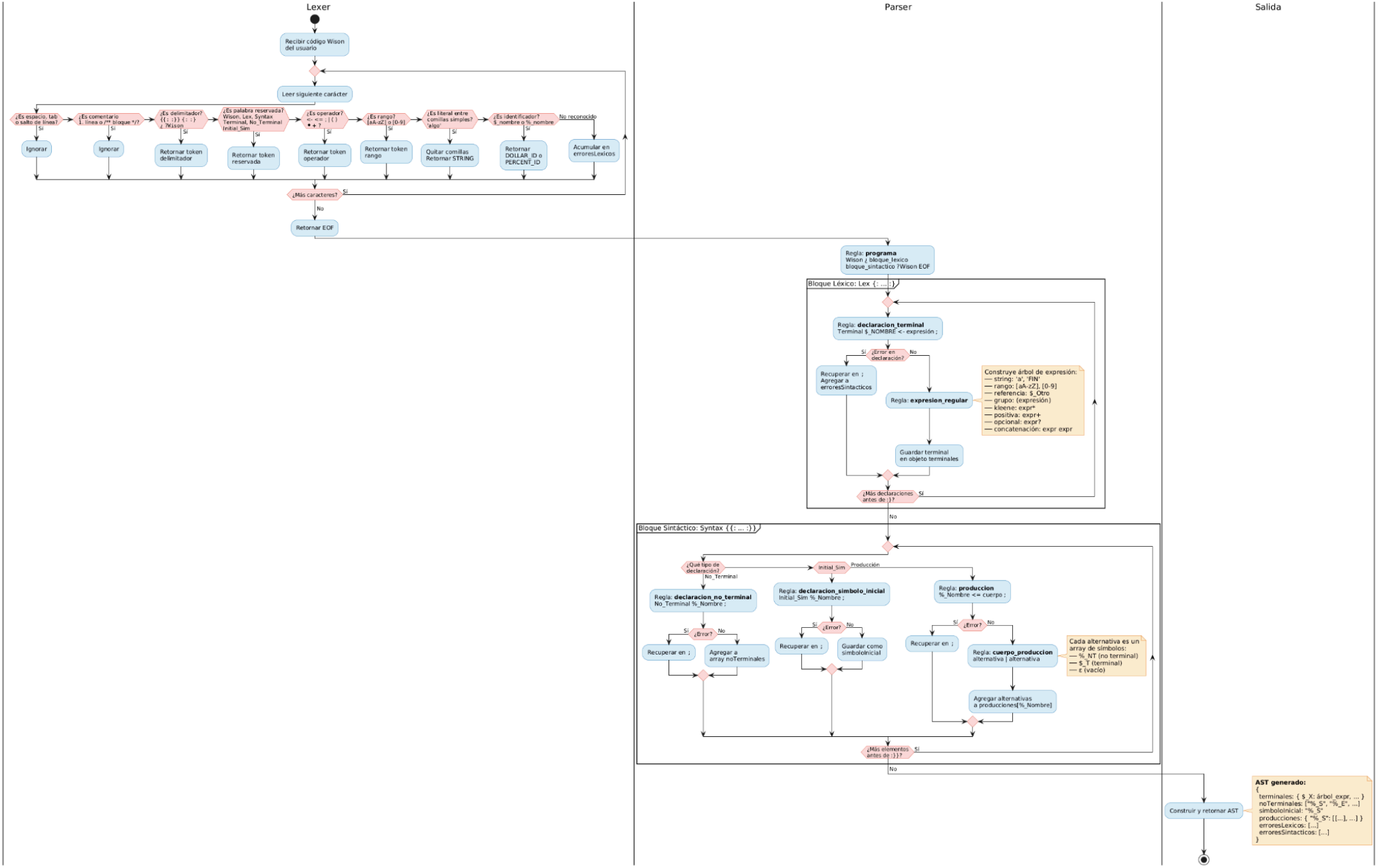
- **terminales**: objeto donde cada clave es el nombre del terminal y el valor es el árbol de expresión que define su patrón léxico.
- **noTerminales**: array con los nombres de los no terminales declarados, en orden de declaración.
- **simboloInicial**: nombre del no terminal definido como símbolo inicial.
- **producciones**: objeto donde cada clave es un no terminal y el valor es un array de alternativas, donde cada alternativa es un array de strings (nombres de terminales y no terminales).
- **erroresLexicos**: array de errores léxicos encontrados durante el parseo del código Wison.
- **erroresSintacticos**: array de errores sintácticos encontrados y recuperados durante el parseo.

Recuperación de errores
El parser implementa recuperación de errores en modo pánico utilizando el token especial `**error**` de Jison (equivalente al `error` de CUP). Los puntos de recuperación son:

- **declaracion_terminal**: si falla algo entre `Terminal` y `;`, descarta hasta encontrar `;` y reporta el error.
- **declaracion_no_terminal**: si falla algo entre `No_Terminal` y `;`, recupera en `;`.

- **declaracion_simbolo_inicial**: si falla algo entre `Initial_Sim` y `;`, recupera en `;`.
- **produccion**: si falla algo entre `%_Nombre` y `;`, recupera en `;`.
- **programa**: si falta `?Wison` al final, captura el error antes del EOF.

Esto permite que el parser continúe analizando después de un error y reporte múltiples errores en una sola pasada, en lugar de detenerse en el primer error encontrado.


Lexer
Recibir código Wison del usuario
Leer siguiente carácter
¿Es espacio, tab o salto de línea?
Sí
Ignorar
¿Es comentario (_linea o /** bloque */)?
Sí
Ignorar
¿Es delimitador? ({ : ;}) { : ; } ¿ /Wison
Sí
Retornar token delimitador
¿Es palabra reservada? Wison, Lex, Syntax Terminal, No_Terminal Initial_Sim
Sí
Retornar token reservada
¿Es operador? <- <= ; | ( ) * + ?
Sí
Retornar token operador
¿Es rango? [aA-zZ] o [0-9]
Sí
Retornar token rango
¿Es literal entre comillas simples? 'algo'
Sí
Quitar comillas Retornar STRING
¿Es identificador? $_nombre o %_nombre
Sí
Retornar DOLLAR_ID o PERCENT_ID
No reconocido
Acumular en erroresLexicos
¿Más caracteres?
Sí
No
Retornar EOF
Parser
Regla: programa
Wison ¿ bloque_lexico
bloque_sintactico ?Wison EOF
Bloque Léxico: Lex {: ... :}
Regla: declaracion_terminal
Terminal $_NOMBRE <- expresión ;
¿Error en declaración?
Sí
Recuperar en ; Agregar a erroresSintacticos
No
Regla: expresion_regular
Construye árbol de expresión:
— string: 'a', 'FIN'
— rango: [aA-zZ], [0-9]
— referencia: $_Otro
— grupo: (expresión)
— kleene: expr*
— positiva: expr+
— opcional: expr?
— concatenación: expr expr
Guardar terminal en objeto terminales
¿Más declaraciones antes de :}?
Sí
No
Bloque Sintáctico: Syntax {{: ... :}}
¿Qué tipo de declaración?
No_Terminal
Regla: declaracion_no_terminal
No_Terminal %_Nombre ;
¿Error?
Sí
Recuperar en ;
No
Agregar a array noTerminales
Initial_Sim
Producción
Regla: declaracion_simbolo_inicial
Initial_Sim %_Nombre ;
¿Error?
Sí
Recuperar en ;
No
Guardar como simboloInicial
Regla: produccion
%_Nombre <= cuerpo ;
¿Error?
Sí
Recuperar en ;
No
Regla: cuerpo_produccion
alternativa | alternativa
Cada alternativa es un array de símbolos:
— %_NT (no terminal)
— $_T (terminal)
— ε (vacío)
Agregar alternativas a producciones[%_Nombre]
¿Más elementos antes de :}}?
Sí
No
Salida
Construir y retornar AST
AST generado:
{
terminales: { $_X: árbol_expr, ... }
noTerminales: ["%_S", "%_E", ...]
simboloInicial: "%_S"
producciones: { "%_S": [[...], ...] }
erroresLexicos: [...]
erroresSintacticos: [...]
}

Motor LL1

El motor LL(1) es el módulo central del proyecto. Recibe el AST generado por el parser de Wison y ejecuta los tres pasos fundamentales del análisis sintáctico descendente: calcular los conjuntos **PRIMERO (First)**, calcular los conjuntos **SIGUIENTE (Follow)**, y construir la **Tabla de Análisis Sintáctico**. Si la tabla presenta colisiones (dos producciones en la misma celda), la gramática no es LL(1) y se reporta el error.
El módulo está compuesto por cuatro clases, cada una con una responsabilidad específica:

CalculadorPrimero.js — calcula PRIMERO para cada no terminal
CalculadorSiguiente.js — calcula SIGUIENTE para cada no terminal
ConstructorTablaM.js — construye la tabla de análisis sintáctico
MotorLL1.js — orquesta las tres clases anteriores

CalculadorPrimero
Propósito: Calcular el conjunto PRIMERO para cada no terminal de la gramática.
PRIMERO(X) es el conjunto de terminales que pueden aparecer como primer símbolo de alguna cadena derivada a partir de X.
Datos de entrada:
producciones: objeto con las producciones de la gramática extraídas del AST
noTerminales: array con los nombres de todos los no terminales declarados

Dato de salida:
Objeto donde cada clave es un no terminal y cada valor es un Set con los terminales que pertenecen a su conjunto PRIMERO

El cálculo se realiza de forma iterativa hasta alcanzar un punto fijo, es decir, hasta que no se agreguen más elementos a ningún conjunto PRIMERO. Esto es necesario porque los conjuntos PRIMERO dependen unos de otros cuando hay producciones con no terminales en el lado derecho.
Las reglas aplicadas son las siguientes:
Regla 1: Si X es un terminal, entonces PRIMERO(X) = {X}. Esta regla se aplica implícitamente cuando se encuentra un terminal al inicio de una alternativa.
Regla 2: Si X es un no terminal y X → Y1 Y2 ... Yk es una producción, se recorren los símbolos Y1, Y2, etc. de izquierda a derecha. Se agrega PRIMERO(Y1) sin épsilon a PRIMERO(X). Si Y1 puede derivar en épsilon, se continúa con Y2 y se agrega PRIMERO(Y2) sin épsilon, y así sucesivamente. Si todos los Yi pueden derivar en épsilon, se agrega épsilon a PRIMERO(X).
Regla 3: Si X → ε es una producción, se agrega ε a PRIMERO(X).
Método estático auxiliar: primeroDeSecuencia
Además del cálculo principal, la clase expone un método estático primeroDeSecuencia(secuencia, primero) que calcula PRIMERO de una secuencia de símbolos (no de un solo no terminal). Este método es utilizado por CalculadorSiguiente y ConstructorTablaM para calcular PRIMERO del cuerpo de una producción.
Ejemplo de ejecución:
Dada la gramática:

%_S → %_E $_FIN
%_E → $_A %_Ep
%_Ep → $_Mas $_A %_Ep | ε
El resultado sería:
PRIMERO(%_S) = { $_A }
PRIMERO(%_E) = { $_A }
PRIMERO(%_Ep) = { $_Mas, ε }
Explicación paso a paso:

%_Ep: tiene dos alternativas. La primera empieza con $_Mas (terminal), entonces se agrega $_Mas. La segunda es ε, entonces se agrega ε.
%_E: empieza con $_A que es terminal, se agrega $_A y se detiene.
%_S: empieza con %_E que es no terminal. PRIMERO(%_E) = {_A}, que no contiene ε, entonces PRIMERO(%_S) = {
_A}.

CalculadorSiguiente
Propósito: Calcular el conjunto SIGUIENTE para cada no terminal de la gramática. SIGUIENTE(A) es el conjunto de terminales que pueden aparecer inmediatamente a la derecha de A en alguna forma de frase derivada desde el símbolo inicial.
Datos de entrada:
producciones: objeto con las producciones de la gramática
noTerminales: array con los nombres de los no terminales
simboloInicial: nombre del no terminal que es símbolo inicial
primero: objeto con los conjuntos PRIMERO ya calculados por CalculadorPrimero

Dato de salida:
Objeto donde cada clave es un no terminal y cada valor es un Set con los terminales que pertenecen a su conjunto SIGUIENTE

Al igual que PRIMERO, el cálculo se realiza iterativamente hasta que no haya cambios. Las reglas aplicadas son:
Regla 1: Se agrega $ (fin de cadena) a SIGUIENTE del símbolo inicial. El símbolo $ representa el final de la entrada y siempre puede aparecer después del símbolo inicial.
Regla 2: Si existe una producción A → αBβ (donde B es un no terminal y β es lo que sigue después de B), se agrega PRIMERO(β) sin épsilon a SIGUIENTE(B). Esto es porque lo que puede aparecer al inicio de β es lo que puede seguir inmediatamente a B en ese contexto.
Regla 3: Si existe una producción A → αB (B está al final), o A → αBβ donde β puede derivar en épsilon, se agrega todo SIGUIENTE(A) a SIGUIENTE(B). La lógica es: si B está al final de lo que produce A, entonces lo que pueda seguir a A también puede seguir a B.
Dependencia: Esta clase utiliza el método estático CalculadorPrimero.primeroDeSecuencia para calcular PRIMERO de β (lo que sigue después de un no terminal en una producción).
Ejemplo de ejecución:
Continuando con la misma gramática:
%_S → %_E $_FIN
%_E → $_A %_Ep
%_Ep → $_Mas $_A %_Ep | ε
El resultado sería:

SIGUIENTE(%_S) = { $ }
SIGUIENTE(%_E) = { $_FIN }
SIGUIENTE(%_Ep) = { $_FIN }
Explicación paso a paso:

%_S es el símbolo inicial, se agrega $ (Regla 1).
En %_S → %_E $_FIN: después de %_E viene $_FIN. PRIMERO(_FIN) = {
_FIN}. Se agrega
$_FIN a SIGUIENTE(%_E) (Regla 2).
En %_E → $_A %_Ep: %_Ep está al final de la producción. Se agrega SIGUIENTE(%_E) = {$_FIN} a SIGUIENTE(%_Ep) (Regla 3).
En %_Ep → $_Mas $_A %_Ep: %_Ep está al final. Se agrega SIGUIENTE(%_Ep) a SIGUIENTE(%_Ep), que ya tiene {$_FIN}, no cambia nada.

ConstructorTablaM
Propósito: Construir la tabla de análisis sintáctico (Tabla M) a partir de los conjuntos PRIMERO y SIGUIENTE. La Tabla M es una matriz bidimensional indexada por [no terminal, terminal] que indica qué producción aplicar cuando el parser está en un no terminal y ve un terminal en la entrada.
Datos de entrada:
producciones: objeto con las producciones de la gramática
noTerminales: array con los nombres de los no terminales
terminales: objeto con los terminales de la gramática
primero: conjuntos PRIMERO calculados
siguiente: conjuntos SIGUIENTE calculados

Datos de salida:
tabla: objeto bidimensional donde tabla[noTerminal][terminal] contiene el array de símbolos de la producción a aplicar
colisiones: array de errores indicando las celdas donde hay más de una producción (la gramática no es LL(1))

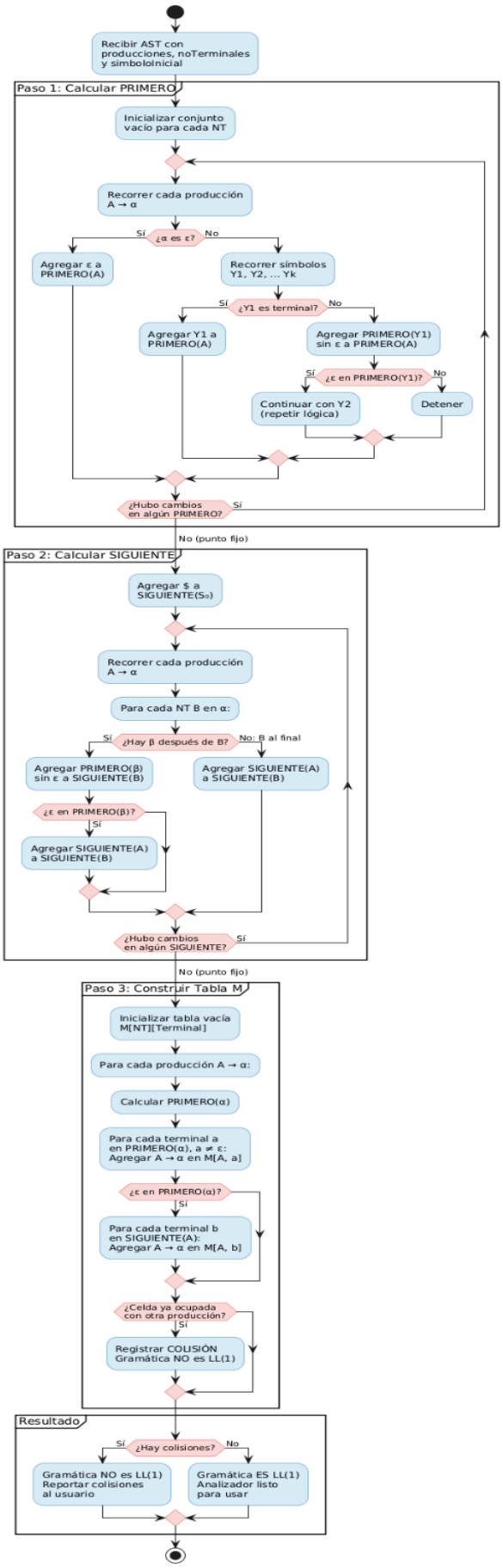
Para cada producción A → α de la gramática:
Regla 1: Se calcula PRIMERO(α) (PRIMERO de toda la alternativa). Para cada terminal a en PRIMERO(α) que no sea ε, se agrega la producción A → α en la celda M[A, a].
Regla 2: Si ε está en PRIMERO(α), significa que la alternativa puede derivar en vacío. En ese caso, para cada terminal b en SIGUIENTE(A), se agrega la producción A → α en la celda M[A, b].
Detección de colisiones: Antes de agregar una producción a una celda, se verifica si la celda ya tiene una producción distinta. Si la tiene, se genera un error de colisión indicando qué celda tiene el conflicto y cuáles son las dos producciones involucradas. Una colisión significa que la gramática no es LL(1) — el parser no puede decidir de forma determinista qué producción aplicar.
Causas comunes de colisiones:
Recursividad por la izquierda: una producción como A → A b genera que PRIMERO(A) se use en sí mismo, creando conflictos.

Falta de factorización: dos alternativas del mismo no terminal que empiezan con el mismo terminal, como A → a b | a c.
Ambigüedad: cuando la gramática permite más de un árbol de derivación para la misma cadena.

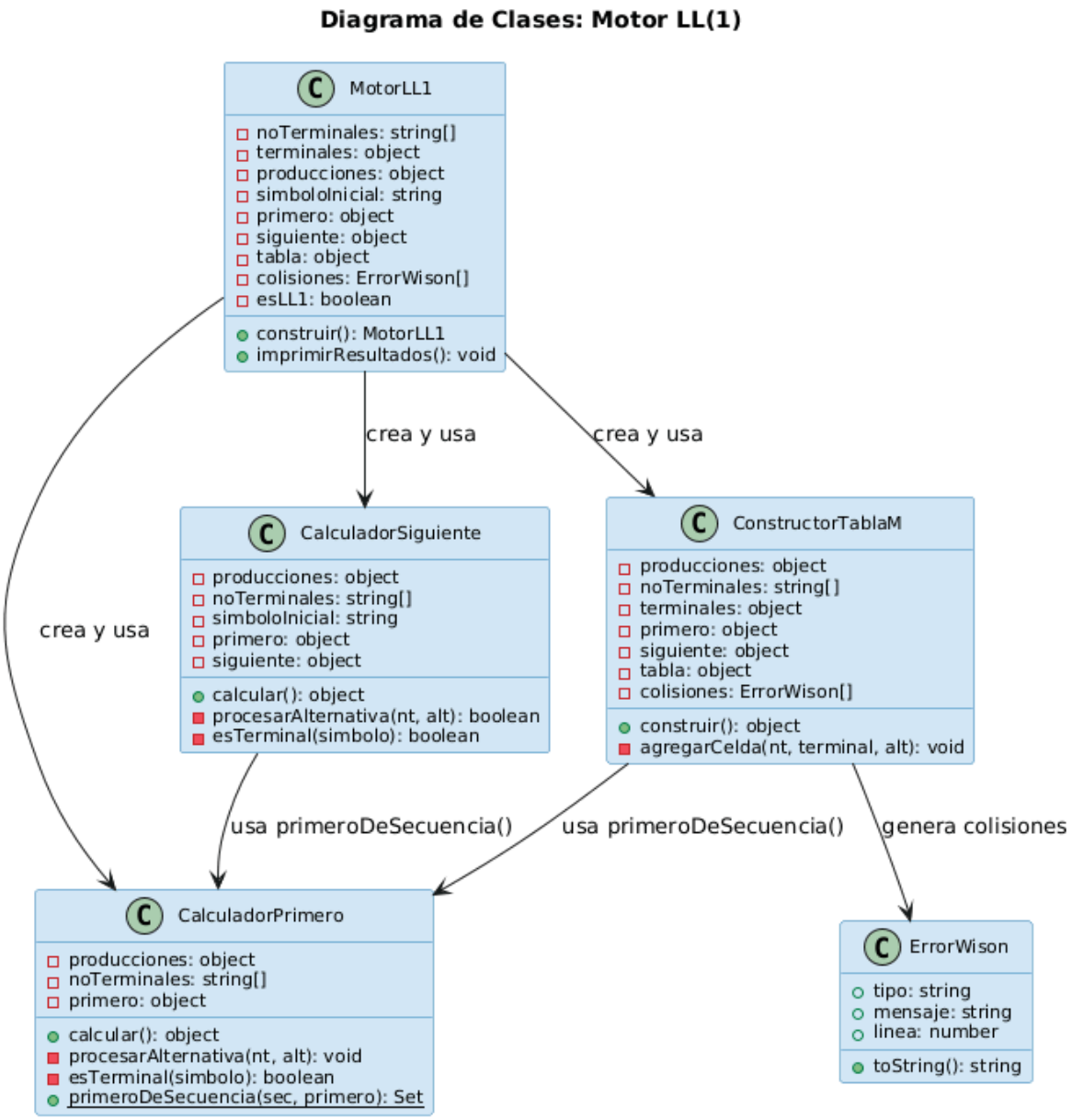
MotorLL1
Propósito: Clase orquestadora que ejecuta todo el proceso LL(1) en secuencia. Es el punto de entrada para la fase 4 del sistema.
Datos de entrada:
ast: el AST completo generado por el parser de Wison

Datos de salida (propiedades de la instancia):
primero: los conjuntos PRIMERO calculados
siguiente: los conjuntos SIGUIENTE calculados
tabla: la tabla M construida
colisiones: array de colisiones encontradas
esLL1: booleano que indica si la gramática es LL(1) (sin colisiones)

Flujo de ejecución:
Al llamar al método construir():
Paso 1 — Crear una instancia de CalculadorPrimero con las producciones y no terminales del AST. Llamar a su método calcular() para obtener los conjuntos PRIMERO.
Paso 2 — Crear una instancia de CalculadorSiguiente con las producciones, no terminales, símbolo inicial y los conjuntos PRIMERO recién calculados. Llamar a su método calcular() para obtener los conjuntos SIGUIENTE.
Paso 3 — Crear una instancia de ConstructorTablaM con las producciones, no terminales, terminales, conjuntos PRIMERO y conjuntos SIGUIENTE. Llamar a su método construir() para obtener la tabla M y las colisiones.
Paso 4 — Determinar si la gramática es LL(1) verificando que no haya colisiones.


Recibir AST con producciones, noTerminales y simbololnicial
Paso 1: Calcular PRIMERO
Inicializar conjunto vacío para cada NT
Recorrer cada producción A → α
¿α es ε?
Sí
No
Agregar ε a PRIMERO(A)
Recorrer símbolos Y1, Y2, ... Yk
¿Y1 es terminal?
Agregar Y1 a PRIMERO(A)
Agregar PRIMERO(Y1) sin ε a PRIMERO(A)
¿ε en PRIMERO(Y1)?
Continuar con Y2 (repetir lógica)
Detener
¿Hubo cambios en algún PRIMERO?
No (punto fijo)
Paso 2: Calcular SIGUIENTE
Agregar $ a SIGUIENTE(S₀)
Recorrer cada producción A → α
Para cada NT B en α:
¿Hay β después de B?
No: B al final
Agregar PRIMERO(β) sin ε a SIGUIENTE(B)
Agregar SIGUIENTE(A) a SIGUIENTE(B)
¿ε en PRIMERO(β)?
Agregar SIGUIENTE(A) a SIGUIENTE(B)
¿Hubo cambios en algún SIGUIENTE?
No (punto fijo)
Paso 3: Construir Tabla M
Inicializar tabla vacía M[NT][Terminal]
Para cada producción A → α:
Calcular PRIMERO(α)
Para cada terminal a en PRIMERO(α), a ≠ ε: Agregar A → α en M[A, a]
¿ε en PRIMERO(α)?
Para cada terminal b en SIGUIENTE(A): Agregar A → α en M[A, b]
¿Celda ya ocupada con otra producción?
Registrar COLISIÓN Gramática NO es LL(1)
Resultado
¿Hay colisiones?
Gramática NO es LL(1) Reportar colisiones al usuario
Gramática ES LL(1) Analizador listo para usar

relacion entre clases



Lexer:

El lexer dinámico es el módulo encargado de tokenizar las cadenas de entrada del usuario utilizando los terminales definidos en el bloque Lex del código Wison. A diferencia de un lexer tradicional donde los tokens están predefinidos en el código fuente, este lexer se construye en tiempo de ejecución a partir de las definiciones que el usuario escribió.
El módulo está compuesto por tres clases que trabajan en secuencia:
ResolverReferencias.js — resuelve las referencias entre terminales
ConstructorRegex.js — convierte los árboles de expresión a expresiones regulares de JavaScript
Tokenizador.js — usa las expresiones regulares para tokenizar cadenas de entrada
El flujo es: el AST contiene un árbol de expresión por cada terminal. Primero se resuelven las referencias cruzadas entre terminales, después se convierten los árboles a expresiones regulares nativas de JavaScript, y finalmente se usa el motor de regex de JavaScript para tokenizar.

ResolverReferencias
Propósito: Resolver las referencias entre terminales. En Wison un terminal puede referenciar a otro usando la sintaxis ($_Nombre). Por ejemplo:
Terminal $_Punto <- '.' ;
Terminal $_NUMEROS_2 <- [0-9]+ ;
Terminal $_Decimal <- ([0-9]*)($_Punto)($_NUMEROS_2) ;
Acá $_Decimal referencia a $_Punto y a $_NUMEROS_2. Esta clase reemplaza esas referencias por los árboles de expresión reales de los terminales referenciados.
Datos de entrada:
terminales: objeto del AST donde cada clave es el nombre del terminal y cada valor es su árbol de expresión
Dato de salida:
Nuevo objeto de terminales con todas las referencias reemplazadas por los árboles reales

La resolución se hace de forma recursiva. Para cada nodo del árbol de expresión:
Si es un nodo de tipo referencia, se busca el terminal referenciado y se reemplaza por una copia profunda de su árbol de expresión. La copia profunda (usando JSON.parse(JSON.stringify(...))) es necesaria para que los terminales no compartan nodos en memoria.
Si es un nodo de tipo concatenacion, se resuelven recursivamente los hijos izquierdo y derecho.
Si es un nodo de tipo kleene, positiva u opcional, se resuelve recursivamente la subexpresión.
Si es un nodo de tipo grupo, se desenvuelve (los paréntesis ya cumplieron su función en el parser) y se resuelve la expresión interna.
Si es un nodo de tipo string o rango, se retorna una copia tal cual ya que son nodos hoja sin referencias.

ConstructorRegex
Propósito: Convertir los árboles de expresión de los terminales (ya con referencias resueltas) a expresiones regulares nativas de JavaScript (objetos RegExp).
Datos de entrada:
terminales: objeto del AST con los terminales y sus árboles de expresión
Dato de salida:
Objeto donde cada clave es el nombre del terminal y cada valor es un objeto RegExp de JavaScript
La clase utiliza ResolverReferencias internamente para resolver las referencias antes de convertir. Luego recorre cada árbol de expresión de forma recursiva y genera un string de expresión regular:

Nodo string: el texto se escapa para que los caracteres especiales de regex no se interpreten. Por ejemplo, el punto "." se convierte a "\." y el más "+" se convierte a "\+". La función de escape usa texto.replace(/[.*+?^${}()|[\]\\]/g, '\\$&').
Nodo rango: se convierte directamente a la clase de caracteres de JavaScript equivalente. aA-zZ se convierte a [a-zA-Z] y 0-9 se convierte a [0-9].
Nodo concatenacion: se concatenan los strings de regex del hijo izquierdo y el hijo derecho. No se necesita un operador explícito ya que en regex la concatenación es implícita.
Nodo kleene: se envuelve la subexpresión en un grupo no capturador con el operador asterisco: (?:subexpresión)*.
Nodo positiva: igual que kleene pero con el operador más: (?:subexpresión)+.
Nodo opcional: igual pero con el operador interrogación: (?:subexpresión)?.
Nodo grupo: se envuelve en grupo no capturador: (?:subexpresión).
Nodo referencia: si llega un nodo de referencia sin resolver, genera string vacío. Esto no debería ocurrir si ResolverReferencias se ejecutó correctamente.

Terminal	Árbol	Regex generada
`$_A <- 'a'`	string("a")	`/^(?:a)/`
`$_Mas <- '+'`	string("+")	`/^(?:\+)/`
`$_FIN <- 'FIN'`	string("FIN")	`/^(?:FIN)/`
`$_NUM <- [0-9]+`	positiva(rango(0-9))	`/^(?:(?:[0-9])+)/`
`$_ID <- ([aA-zZ])([aA-zZ]*)`	concat(rango(aA-zZ), kleene(rango(aA-zZ)))	`/^(?:[a-zA-Z](?:[a-zA-Z])*)/`
`$_DEC <- ([0-9]+)($_P)([0-9]+)`	concat(concat(positiva(rango(0-9)), string(".")), positiva(rango(0-9)))	`/^(?:(?:[0-9])+\.(?:[0-9])+)/`

Tokenizador
Propósito: Recibir una cadena de texto y convertirla en una lista de tokens usando las expresiones regulares generadas por ConstructorRegex. Implementa la estrategia de longest match (coincidencia más larga) con un sistema de prioridades para resolver empates.
Datos de entrada (constructor):
terminales: objeto del AST con los terminales y sus árboles de expresión
Datos de entrada (método tokenizar):
texto: string con la cadena a tokenizar
Datos de salida:
tokens: array de objetos con tipo (nombre del terminal), valor (texto matcheado) y posicion (posición en la cadena original)
errores: array de errores léxicos (caracteres no reconocidos)

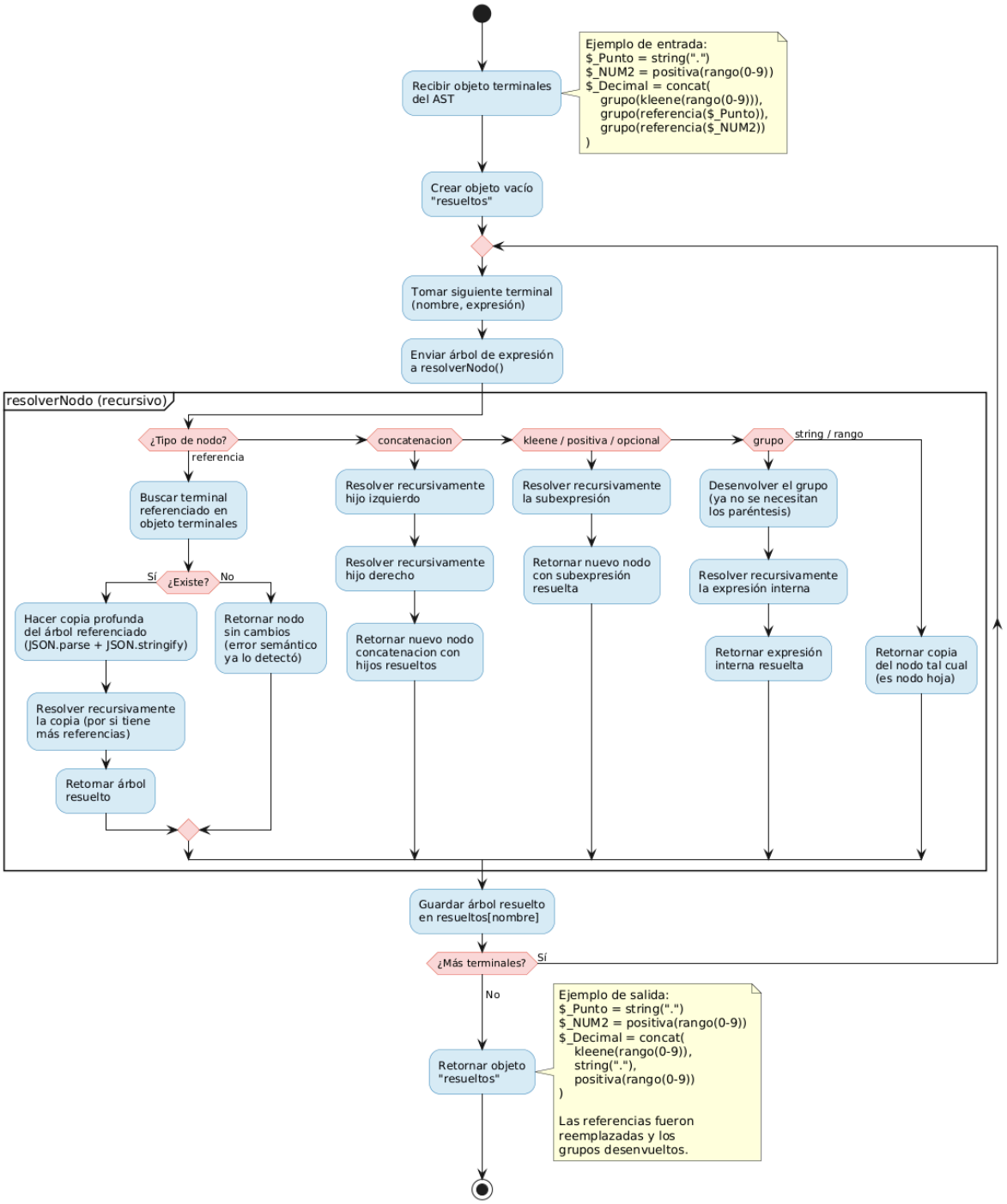
Sistema de prioridades:
Cuando dos terminales matchean la misma cantidad de caracteres desde la misma posición, se necesita un criterio de desempate. El tokenizador asigna una prioridad numérica a cada terminal según la complejidad de su definición:
Prioridad 3 (Literal): terminales definidos con un string puro, como $_INT <- 'int' o $_SUMA <- '+'. Son los más específicos porque matchean texto exacto.
Prioridad 2 (Compuesto): terminales con concatenación o referencias, como $_ID <- ([aA-zZ])([aA-zZ]*). Son patrones más elaborados que combinan múltiples elementos.
Prioridad 1 (Simple): terminales con un rango solo, como $_LETRA <- [aA-zZ] o $_DIGITO <- [0-9]. Son los más genéricos.

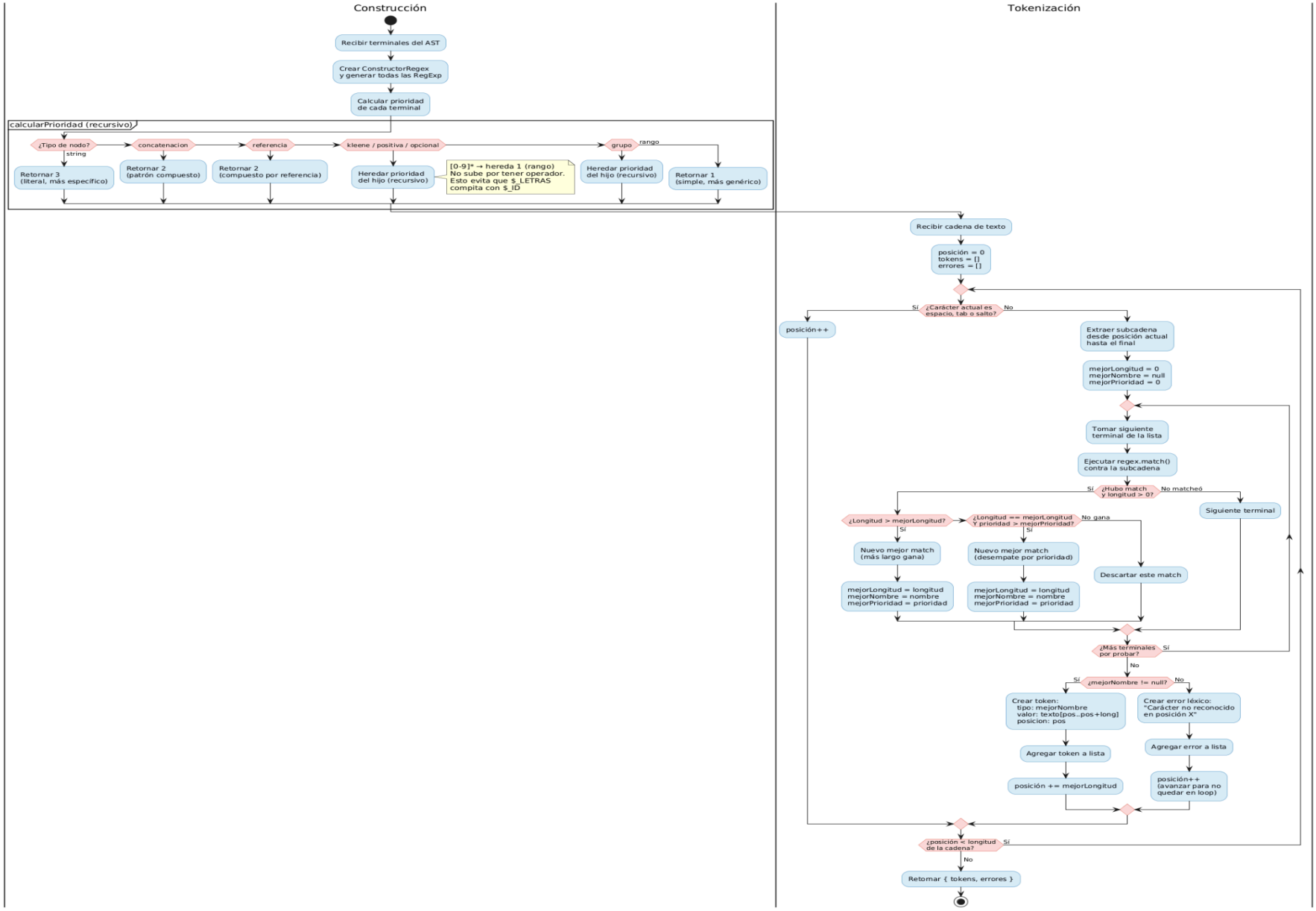
La prioridad se calcula recursivamente: los operadores *, + y ? heredan la prioridad de su hijo (un kleene sobre un rango sigue siendo prioridad 1), mientras que la concatenación siempre sube a prioridad 2.

tokenización:
El tokenizador recorre la cadena de izquierda a derecha, posición por posición:

1. Si el carácter actual es un espacio, tabulación o salto de línea, se salta y se avanza a la siguiente posición.
2. Se extrae la subcadena desde la posición actual hasta el final del texto.
3. Se prueba cada expresión regular contra el inicio de la subcadena. Para cada match exitoso se registra la longitud del match, el nombre del terminal y su prioridad.
4. Se elige el ganador siguiendo dos criterios en orden: primero el match más largo (longest match), y en caso de empate en longitud, el de mayor prioridad.
5. Si hay un ganador, se crea un token con el tipo (nombre del terminal), el valor (texto matcheado) y la posición en la cadena original. Se avanza la posición según la longitud del match.
6. Si ningún terminal matchea, se registra un error léxico con el carácter no reconocido y se avanza una posición para continuar el análisis.

Diagrama de Flujo: ResolverReferencias
Resolución de referencias entre terminales




Construcción
Recibir terminales del AST
Crear ConstructorRegex
y generar todas las RegExp
Calcular prioridad
de cada terminal
calcularPrioridad (recursivo)
¿Tipo de nodo?
string
Retornar 3
(literal, más específico)
concatenacion
Retornar 2
(patrón compuesto)
referencia
Retornar 2
(compuesto por referencia)
kleene / positiva / opcional
Heredar prioridad
del hijo (recursivo)
[0-9]* → hereda 1 (rango)
No sube por tener operador.
Esto evita que $_LETRAS
compita con $_ID
grupo
Heredar prioridad
del hijo (recursivo)
rango
Retornar 1
(simple, más genérico)
Tokenización
Recibir cadena de texto
posición = 0
tokens = []
errores = []
¿Carácter actual es
espacio, tab o salto?
Sí
No
posición++
Extraer subcadena
desde posición actual
hasta el final
mejorLongitud = 0
mejorNombre = null
mejorPrioridad = 0
Tomar siguiente
terminal de la lista
Ejecutar regex.match()
contra la subcadena
¿Hubo match
y longitud > 0?
No matcheó
Siguiente terminal
¿Longitud > mejorLongitud?
¿Longitud == mejorLongitud
Y prioridad > mejorPrioridad?
No gana
Nuevo mejor match
(más largo gana)
Nuevo mejor match
(desempate por prioridad)
Descartar este match
mejorLongitud = longitud
mejorNombre = nombre
mejorPrioridad = prioridad
¿Más terminales
por probar?
¿mejorNombre != null?
Crear token:
tipo: mejorNombre
valor: texto[pos..pos+long]
posicion: pos
Crear error léxico:
"Carácter no reconocido
en posición X"
Agregar token a lista
Agregar error a lista
posición += mejorLongitud
posición++
(avanzar para no
quedar en loop)
¿posición < longitud
de la cadena?
Retomar { tokens, errores }

Parser Descendente

El parser descendente recursivo es el módulo que ejecuta el análisis sintáctico de las cadenas de entrada del usuario. Recibe una lista de tokens (producida por el Tokenizador) y utiliza la Tabla M (construida por el MotorLL1) para verificar si la secuencia de tokens es válida según la gramática definida por el usuario. Si la cadena es aceptada, genera el árbol de derivación. Si es rechazada, reporta errores indicando qué se esperaba y qué se encontró.
Este módulo es la implementación d donde cada no terminal tiene una función asociada que consulta la Tabla M para decidir qué producción aplicar.
El módulo está compuesto por dos clases:

NodoArbol.js — representa cada nodo del árbol de derivación
ParserDescendente.js — ejecuta el análisis y construye el árbol

NodoArbol
Propósito: Representar un nodo en el árbol de derivación. Cada nodo puede ser un no terminal (nodo interno del árbol) o un terminal (nodo hoja del árbol).
Propiedades:

nombre: string con el nombre del símbolo. Para terminales es algo como $_A, $_Mas. Para no terminales es algo como %_S, %_Ep. Para épsilon es ε.
esTerminal: booleano que indica si el nodo es terminal (true) o no terminal (false).
valor: string con el valor del token consumido. Solo se llena para terminales — por ejemplo, si el terminal $_A consumió el carácter a, el valor es "a". Para no terminales es null.
hijos: array de objetos NodoArbol. Los no terminales tienen hijos (los símbolos de la producción que se aplicó). Los terminales no tienen hijos (son hojas).

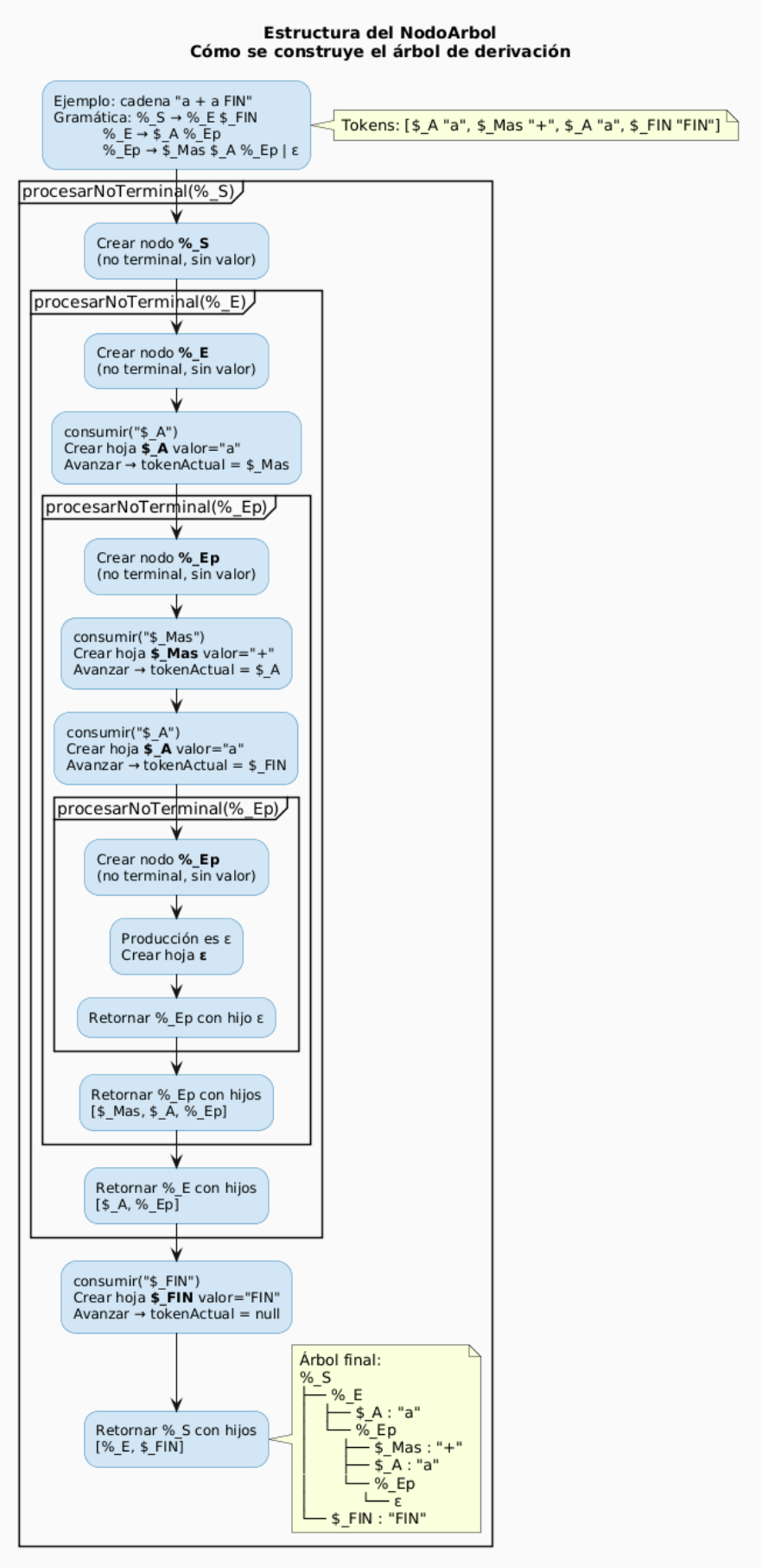
Método:
agregarHijo(hijo): agrega un NodoArbol al array de hijos. Se usa durante la construcción del árbol conforme el parser va procesando los símbolos de cada producción.

```
NodoArbol(%_S, noTerminal)
├── NodoArbol(%_E, noTerminal)
│   ├── NodoArbol($_A, terminal, valor="a")
│   └── NodoArbol(%_Ep, noTerminal)
│       ├── NodoArbol($_Mas, terminal, valor="+")
│       ├── NodoArbol($_A, terminal, valor="a")
│       └── NodoArbol(%_Ep, noTerminal)
│           └── NodoArbol(ε, terminal, valor="ε")
└── NodoArbol($_FIN, terminal, valor="FIN")
```

ParserDescendente
procesarNoTerminal(nombre) que consulta la Tabla M dinámicamente para cualquier gramática que el usuario haya definido.
Datos de entrada (constructor):
tablaM: la tabla de análisis sintáctico generada por ConstructorTablaM
simboloInicial: nombre del no terminal que es símbolo inicial
Datos de entrada (método analizar):
tokens: array de tokens generados por el Tokenizador
Datos de salida:
arbol: objeto NodoArbol con la raíz del árbol de derivación
errores: array de errores sintácticos encontrados
aceptada: booleano que indica si la cadena fue aceptada

Propiedades internas durante el análisis:
tokens: la lista completa de tokens a analizar
posicion: índice actual en la lista de tokens (empieza en 0)
tokenActual: el token que se está mirando actualmente (equivalente a la variable global token del pseudocódigo)
errores: array donde se acumulan los errores

main() → método **analizar(tokens)**: inicializa el estado y llama a procesarNoTerminal(simboloInicial)
token **= getToken()** → método avanzar(): mueve posicion al siguiente token y actualiza tokenActual
E(), Ep(), T(), F() → método procesarNoTerminal(nombre): recibe cualquier no terminal, consulta la Tabla M y ejecuta la producción correspondiente
switch(token) case X: → consulta tablaM[noTerminal][tokenActual.tipo]: la Tabla M ya tiene precalculado qué producción usar para cada combinación de no terminal y token
consumir(SUMA) → método consumir(tipoEsperado): verifica que el token actual sea el esperado y avanza


Estructura del NodoArbol
Cómo se construye el árbol de derivación
Ejemplo: cadena "a + a FIN"
Gramática: %_S → %_E $_FIN
%_E → $_A %_Ep
%_Ep → $_Mas $_A %_Ep | ε
Tokens: [$_A "a", $_Mas "+", $_A "a", $_FIN "FIN"]
procesarNoTerminal(%_S)
Crear nodo %_S
(no terminal, sin valor)
procesarNoTerminal(%_E)
Crear nodo %_E
(no terminal, sin valor)
consumir("$_A")
Crear hoja $_A valor="a"
Avanzar → tokenActual = $_Mas
procesarNoTerminal(%_Ep)
Crear nodo %_Ep
(no terminal, sin valor)
consumir("$_Mas")
Crear hoja $_Mas valor="+"
Avanzar → tokenActual = $_A
consumir("$_A")
Crear hoja $_A valor="a"
Avanzar → tokenActual = $_FIN
procesarNoTerminal(%_Ep)
Crear nodo %_Ep
(no terminal, sin valor)
Producción es ε
Crear hoja ε
Retornar %_Ep con hijo ε
Retornar %_Ep con hijos
[$_Mas, $_A, %_Ep]
Retornar %_E con hijos
[$_A, %_Ep]
consumir("$_FIN")
Crear hoja $_FIN valor="FIN"
Avanzar → tokenActual = null
Retornar %_S con hijos
[%_E, $_FIN]
Árbol final:
%_S
%_E
$_A : "a"
%_Ep
$_Mas : "+"
$_A : "a"
%_Ep
ε
$_FIN : "FIN"